

week_8_lab_airbnb

CHAKAR

2026-09-07

This file cleans the combined Christchurch Airbnb dataset (Oct 2025-Jun 2026): handling missing prices, removing extreme price outliers, and dropping unneeded columns, ready for comparison against the tenancy dataset.

Week 8 instruction for Airbnb file: All_chch listings (separate datasets from oct 2025 to jun 2026 combined, but only for chch city). clean the dataset -> ultimate aim is going to be comparing rental prices and number of available properties with tenancy dataset

```
# EDA decision: prices at/above this are treated as data-entry errors
price_outlier_max <- 5000
```

```
# load required library and file
# load data as "data"
library(tidyverse)

data <- read_csv("../out/all_chch_listings.csv")
```

```
## Rows: 35796 Columns: 19
## -- Column specification -----
## Delimiter: ","
## chr   (6): name, host_name, neighbourhood_group, neighbourhood, room_type, m...
## dbl   (11): id, host_id, latitude, longitude, price, minimum_nights, number_o...
## lgl   (1): license
## date  (1): last_review
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
# summary statistics for all columns
print(summary(data))
```

```
##           id           name      host_id      host_name
## Min.      :2.040e+05 Length   :35796 Min.      :5.738e+04 Length   :35796
## 1st Qu.:5.287e+07  N.unique : 5198 1st Qu.:6.434e+07 N.unique : 1460
## Median :1.104e+18 N.blank  :    0 Median :1.742e+08 N.blank  :    0
## Mean    :9.190e+17 Min.nchar:    3 Mean    :5.452e+15 Min.nchar:    1
## 3rd Qu.:1.415e+18 Max.nchar:   68 3rd Qu.:4.521e+08 Max.nchar:   40
## Max.    :1.750e+18           Max.    :1.750e+18 NAs       :    1
##
## neighbourhood_group neighbourhood latitude longitude
## Length      :35796 Length      :35796 Min.      : -43.87 Min.      :172.4
```

```
## N.unique : 1      N.unique : 16      1st Qu.: -43.56      1st Qu.: 172.6
## N.blank : 0      N.blank : 0      Median : -43.53      Median : 172.6
## Min.nchar: 17     Min.nchar: 10     Mean : -43.56      Mean : 172.7
## Max.nchar: 17     Max.nchar: 20     3rd Qu.: -43.52     3rd Qu.: 172.7
##                                     Max. : -43.42      Max. : 173.1
##
##      room_type      price      minimum_nights      number_of_reviews
## Length :35796      Min. : 27.0      Min. : 1.000      Min. : 0.00
## N.unique : 4      1st Qu.: 145.0      1st Qu.: 1.000      1st Qu.: 5.00
## N.blank : 0      Median : 210.0      Median : 1.000      Median : 26.00
## Min.nchar: 10     Mean : 260.8      Mean : 1.768      Mean : 70.48
## Max.nchar: 15     3rd Qu.: 288.0      3rd Qu.: 2.000      3rd Qu.: 85.00
##                                     Max. : 43654.0      Max. : 365.000      Max. : 1601.00
##                                     NAs : 11318      NAs : 37
##      last_review      reviews_per_month      calculated_host_listings_count
## Min. :2014-04-14      Min. : 0.010      Min. : 1.00
## 1st Qu.:2025-10-27      1st Qu.: 0.750      1st Qu.: 1.00
## Median :2026-02-07      Median : 1.680      Median : 3.00
## Mean :2025-12-01      Mean : 2.324      Mean : 58.01
## 3rd Qu.:2026-04-24      3rd Qu.: 3.280      3rd Qu.: 47.00
## Max. :2026-08-13      Max. : 16.570      Max. : 1586.00
## NAs :3234      NAs : 3234
##      availability_365      number_of_reviews_ltm      license      month_year
## Min. : 0.0      Min. : 0.00      Mode:logical      Length :35796
## 1st Qu.:109.0      1st Qu.: 2.00      NAs :35796      N.unique : 11
## Median :225.0      Median : 10.00      N.blank : 0
## Mean :203.6      Mean : 18.99      Min.nchar: 7
## 3rd Qu.:301.0      3rd Qu.: 27.00      Max.nchar: 7
## Max. :365.0      Max. : 185.00
##
```

data has: Rows: 28795 Columns: 19

```
# datatype
print(glimpse(data))
```

```
## Rows: 35,796
## Columns: 19
## $ id      <dbl> 319770, 737452, 1004252, 1007247, 10974~
## $ name    <chr> "Gateway to the South Island", "Mumford~
## $ host_id  <dbl> 1639584, 3851282, 5523780, 4765250, 499~
## $ host_name <chr> "Jennifer", "Nicky", "Rosie", "John", "~
## $ neighbourhood_group <chr> "Christchurch City", "Christchurch City~
## $ neighbourhood <chr> "Innes Ward", "Banks Peninsula Ward", "~
## $ latitude <dbl> -43.50961, -43.82343, -43.61889, -43.53~
## $ longitude <dbl> 172.6555, 172.9464, 172.6531, 172.6070,~
## $ room_type <chr> "Private room", "Entire home/apt", "Ent~
## $ price    <dbl> 196, 394, 183, 305, 189, 69, 337, 166, ~
## $ minimum_nights <dbl> 1, 2, 2, 2, 1, 5, 3, 2, 2, 2, 1, 2, 2, ~
## $ number_of_reviews <dbl> 3, 102, 227, 119, 230, 205, 16, 181, 98~
## $ last_review <date> 2015-01-09, 2026-04-11, 2026-04-06, 20~
## $ reviews_per_month <dbl> 0.02, 0.68, 1.44, 0.79, 2.05, 1.33, 0.1~
## $ calculated_host_listings_count <dbl> 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 3, 1, 1, ~
```

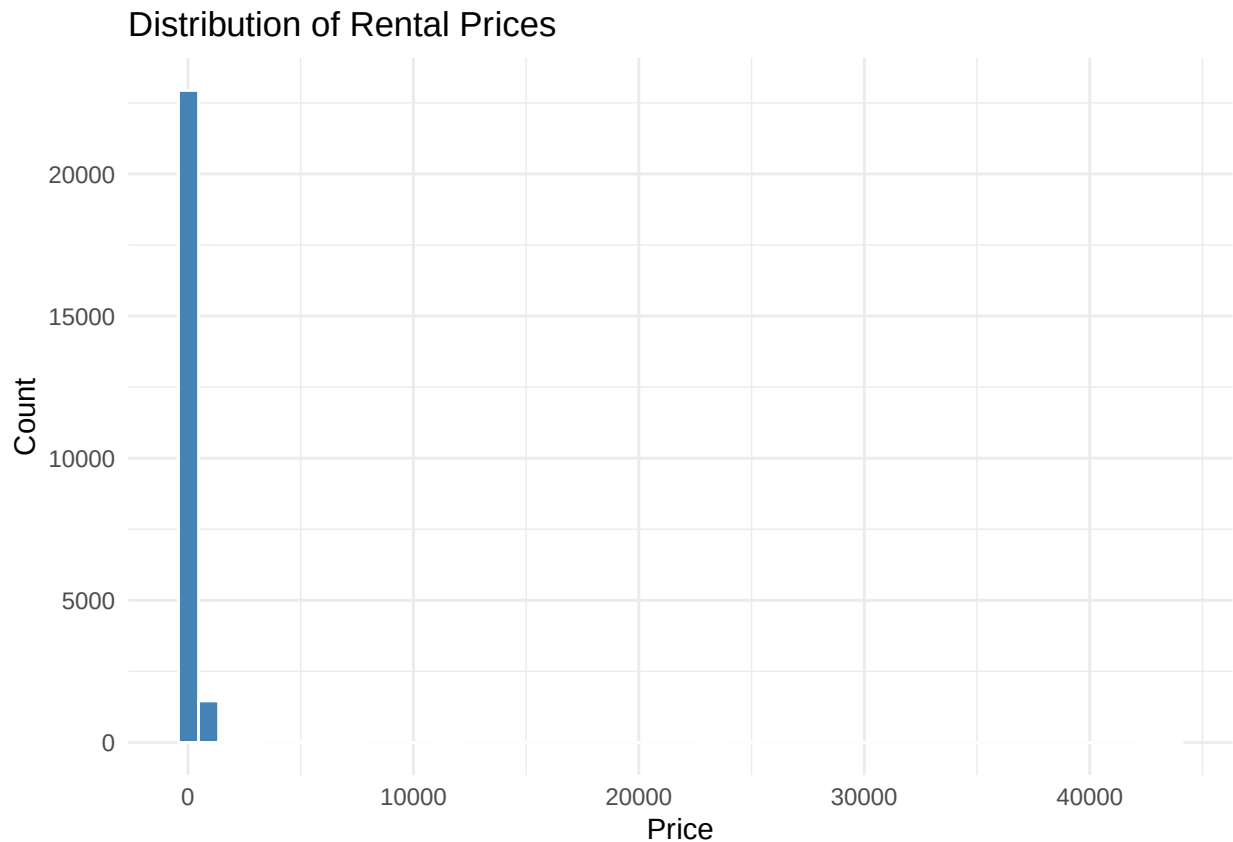
```
## $ availability_365          <dbl> 364, 194, 173, 304, 295, 309, 45, 344, ~
## $ number_of_reviews_ltm    <dbl> 0, 3, 10, 5, 0, 23, 0, 11, 15, 6, 54, 7~
## $ license                  <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ month_year               <chr> "2026-04", "2026-04", "2026-04", "2026--
## # A tibble: 35,796 x 19
##       id name      host_id host_name neighbourhood_group neighbourhood latitude
##       <dbl> <chr>      <dbl> <chr>      <chr>              <chr>          <dbl>
## 1  319770 Gateway~ 1639584 Jennifer Christchurch City Innes Ward      -43.5
## 2  737452 Mumford~ 3851282 Nicky    Christchurch City Banks Penins~   -43.8
## 3  1004252 Governo~ 5523780 Rosie    Christchurch City Banks Penins~   -43.6
## 4  1007247 Clarenc~ 4765250 John     Christchurch City Riccarton Wa~   -43.5
## 5  1097487 Heritag~ 4993285 Paulette  Christchurch City Fendalton Wa~   -43.5
## 6  1429281 Self co~ 7619275 Liz & Go~  Christchurch City Halswell Ward   -43.6
## 7  1461771 Mt Plea~ 6353005 Suzanne  Christchurch City Heathcote Wa~   -43.6
## 8  1798383 3 priva~ 9431529 Shirley  Christchurch City Burwood Ward    -43.5
## 9  1837611 Close t~ 9542449 Shona   Christchurch City Fendalton Wa~   -43.5
## 10 1893596 Luxury ~ 5225682 Shelley  Christchurch City Banks Penins~   -43.6
## # i 35,786 more rows
## # i 12 more variables: longitude <dbl>, room_type <chr>, price <dbl>,
## #   minimum_nights <dbl>, number_of_reviews <dbl>, last_review <date>,
## #   reviews_per_month <dbl>, calculated_host_listings_count <dbl>,
## #   availability_365 <dbl>, number_of_reviews_ltm <dbl>, license <lgl>,
## #   month_year <chr>
```

```
# number of missing NA values
print(colSums(is.na(data)))
```

```
##              id              name
##              0              0
##          host_id          host_name
##              0              1
## neighbourhood_group      neighbourhood
##              0              0
##          latitude          longitude
##              0              0
##          room_type          price
##              0          11318
##          minimum_nights      number_of_reviews
##              37              0
##          last_review      reviews_per_month
##          3234          3234
## calculated_host_listings_count      availability_365
##              0              0
##          number_of_reviews_ltm          license
##              0          35796
##          month_year
##              0
```

```
# plot price distribution
ggplot(data, aes(x = price)) +
  geom_histogram(bins = 50, fill = "steelblue", color = "white") +
  theme_minimal() +
  labs(title = "Distribution of Rental Prices", x = "Price", y = "Count")
```

```
## Warning: Removed 11318 rows containing non-finite outside the scale range
## ('stat_bin()').
```



Instruction from week 5 lab - so probably need days_since_review column - added that column in

```
# days since last reivew (calculate the time difference between scrape/publish date) then plot it
# added scrape_date, days_since_review
data <- data %>%
  mutate(
    scrape_date = ymd(paste0(month_year, "-01")), # Sets scrape date to the 1st of each month
    last_review = ymd(last_review), # Converts the original text column directly into a Date
    days_since_review = as.numeric(scrape_date - last_review)
  )

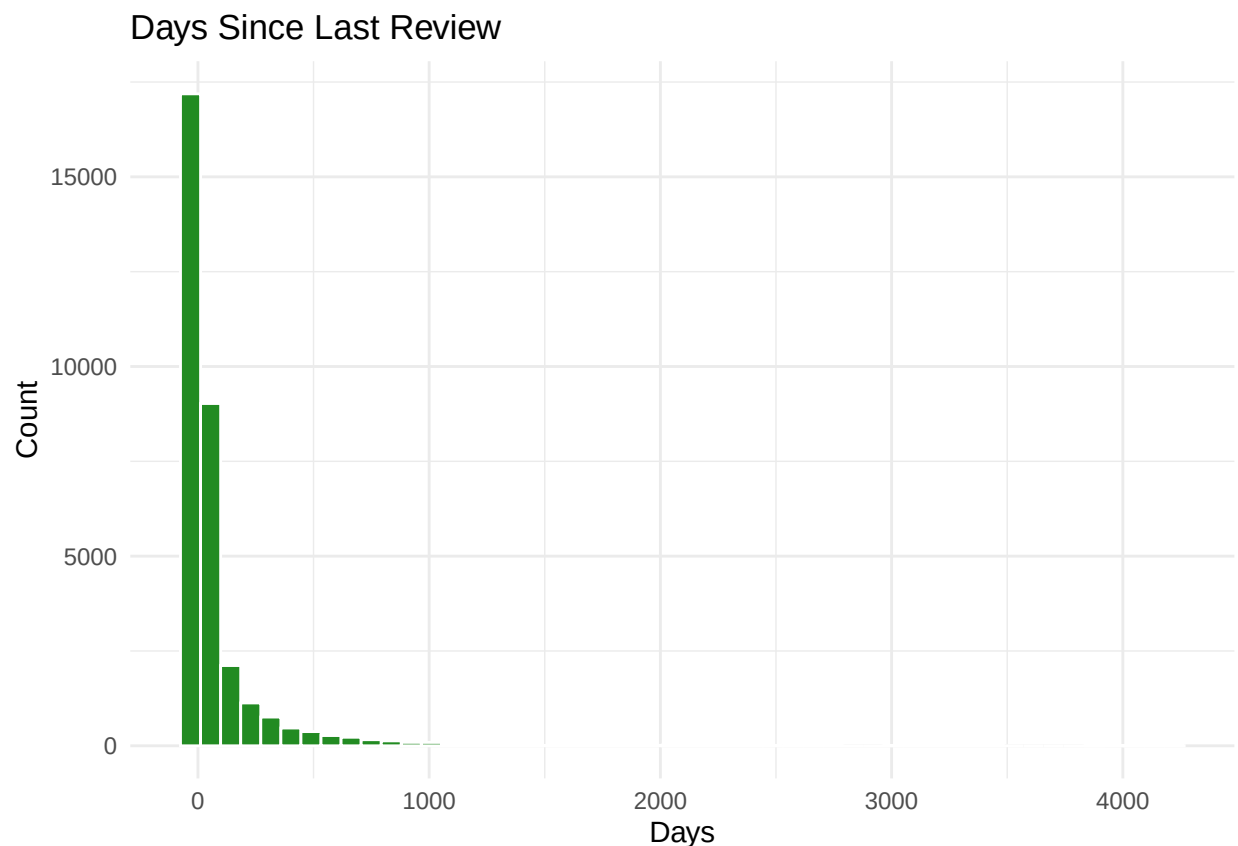
head(data)
```

```
## # A tibble: 6 x 21
##       id name      host_id host_name neighbourhood_group neighbourhood latitude
##   <dbl> <chr>      <dbl> <chr>      <chr>      <chr>      <dbl>
## 1 319770 Gateway ~ 1639584 Jennifer Christchurch City Innes Ward -43.5
## 2 737452 Mumfords~ 3851282 Nicky Christchurch City Banks Penins~ -43.8
## 3 1004252 Governor~ 5523780 Rosie Christchurch City Banks Penins~ -43.6
## 4 1007247 Clarence~ 4765250 John Christchurch City Riccarton Wa~ -43.5
## 5 1097487 Heritage~ 4993285 Paulette Christchurch City Fendalton Wa~ -43.5
## 6 1429281 Self con~ 7619275 Liz & Go~ Christchurch City Halswell Ward -43.6
## # i 14 more variables: longitude <dbl>, room_type <chr>, price <dbl>,
```

```
## # minimum_nights <dbl>, number_of_reviews <dbl>, last_review <date>,
## # reviews_per_month <dbl>, calculated_host_listings_count <dbl>,
## # availability_365 <dbl>, number_of_reviews_ltm <dbl>, license <lgl>,
## # month_year <chr>, scrape_date <date>, days_since_review <dbl>
```

```
ggplot(data, aes(x = days_since_review)) +
  geom_histogram(bins = 50, fill = "forestgreen", color = "white") +
  theme_minimal() +
  labs(title = "Days Since Last Review", x = "Days", y = "Count")
```

```
## Warning: Removed 3234 rows containing non-finite outside the scale range
## ('stat_bin()').
```



```
# Find number of negative days
negative_rows <- data %>%
  filter(days_since_review < 0)
negative_rows
```

```
## # A tibble: 10,899 x 21
##       id name      host_id host_name neighbourhood_group neighbourhood latitude
##   <dbl> <chr>      <dbl> <chr>      <chr> <chr> <dbl>
## 1  737452 Mumford~  3.85e6 Nicky    Christchurch City Banks Penins~ -43.8
## 2 1004252 Governo~  5.52e6 Rosie    Christchurch City Banks Penins~ -43.6
## 3 1429281 Self co~  7.62e6 Liz & Go~ Christchurch City Halswell Ward -43.6
```

```
## 4 2009003 Luxury ~ 1.03e7 Vivienne Christchurch City Banks Penins~ -43.6
## 5 2238148 Private~ 6.16e6 Robyn Christchurch City Papanui Ward -43.5
## 6 2249963 Charmin~ 9.62e5 Gemma Christchurch City Cashmere Ward -43.6
## 7 2352051 Luxury ~ 1.03e7 Vivienne Christchurch City Banks Penins~ -43.6
## 8 3363429 A Perfe~ 1.70e7 Joan Christchurch City Riccarton Wa~ -43.5
## 9 3852872 Yurt in~ 1.30e7 Jane Christchurch City Banks Penins~ -43.8
## 10 3900685 Views H~ 2.02e7 Michael Christchurch City Banks Penins~ -43.6
## # i 10,889 more rows
## # i 14 more variables: longitude <dbl>, room_type <chr>, price <dbl>,
## # minimum_nights <dbl>, number_of_reviews <dbl>, last_review <date>,
## # reviews_per_month <dbl>, calculated_host_listings_count <dbl>,
## # availability_365 <dbl>, number_of_reviews_ltm <dbl>, license <lgl>,
## # month_year <chr>, scrape_date <date>, days_since_review <dbl>
```

Summary statistics: price: median price is \$205, but the max is \$43,654 (a massive extreme outlier? error?).
Price histogram: right skewed, due to extreme price outliers

license: entirely NA (28,795 missing values) -> consider dropping if doesn't need this column

Missing values: price has 10,667 missing values: how to handle? or filter them out?

last_review and reviews_per_month both have 2,627 missing values

days_since_review has 9418 negative values. Days since last review histogram: Most reviews happened very recently (clumped near 0 days), which makes sense for active listings, but there are long-tail stale listings stretching out past 1,000+ days. For month_year column, change values to include the day the scrape date was done (from the website).

```
data_1 <- data %>%
  mutate(
    scrape_date = ceiling_date(ym(month_year), "month") - days(1), # last day of that row's month
    last_review = ymd(last_review),
    days_since_review = as.numeric(scrape_date - last_review)
  )

# Check how many rows in days_since_review still have negative values
negative_rows <- data_1 %>%
  filter(days_since_review < 0)
print(negative_rows)
```

```
## # A tibble: 3 x 21
##       id name      host_id host_name neighbourhood_group neighbourhood latitude
##   <dbl> <chr>      <dbl> <chr>      <chr>          <chr>          <dbl>
## 1 1.68e18 Suite wi~ 1.68e18 Jess      Christchurch City Papanui Ward -43.5
## 2 1.66e18 Apartmen~ 7.56e 8 Felicity Christchurch City Central Ward -43.5
## 3 1.68e18 Architec~ 1.68e18 Paul      Christchurch City Central Ward -43.5
## # i 14 more variables: longitude <dbl>, room_type <chr>, price <dbl>,
## # minimum_nights <dbl>, number_of_reviews <dbl>, last_review <date>,
## # reviews_per_month <dbl>, calculated_host_listings_count <dbl>,
## # availability_365 <dbl>, number_of_reviews_ltm <dbl>, license <lgl>,
## # month_year <chr>, scrape_date <date>, days_since_review <dbl>
```

CHANGED WEEK 10: scrape_date is now the last day of each listing's month

DEAL with price column first -> extreme outliers + 10667 missing values Check Price inconsistencies

```
# Check the data type of the price column
class(data_1$price)
```

```
## [1] "numeric"
```

```
# 3. Check how many missing values (NAs) exist in the price column
sum(is.na(data_1$price))
```

```
## [1] 11318
```

See if 10667 missing prices are random or linked to specific months

```
# Check missing prices grouped by each month scrape
missing_by_month <- data_1 %>%
  group_by(month_year) %>%
  summarise(
    total_listings = n(),
    missing_prices = sum(is.na(price)),
    pct_missing = (missing_prices / total_listings) * 100
  )

print(missing_by_month)
```

```
## # A tibble: 11 x 4
##   month_year total_listings missing_prices pct_missing
##   <chr>          <int>          <int>         <dbl>
## 1 2025-10           2991             170          5.68
## 2 2025-11           3047             212          6.96
## 3 2025-12           3180            3180         100
## 4 2026-01           3057            3057         100
## 5 2026-02           3190            3190         100
## 6 2026-03           3227             168          5.21
## 7 2026-04           3273             204          6.23
## 8 2026-05           3361             212          6.31
## 9 2026-06           3469             274          7.90
## 10 2026-07          3488             315          9.03
## 11 2026-08          3513             336          9.56
```

100% price missing values in 2025-12, 2026-01, 2026-02 months

checking whether a listing has only one unique price across all its available months. If it does, it's stable, can use that price to fill in December, January, and February.

```
# Check if listing names change across months for the same ID
name_check <- data_1 %>%
  filter(!is.na(name)) %>%
  group_by(id) %>%
  summarise(unique_names = n_distinct(name))

# See how many listings ever changed their title
table(name_check$unique_names > 1)
```

```
##
## FALSE TRUE
## 3554 710
```

FALSE (3,535 listings): For the vast majority of properties, the host never changed the listing title across the entire time period you observed. Their title stayed 100% identical.

TRUE (582 listings): For 582 properties, the host did change their listing title at least once during your dataset

```
# 1. Check price consistency for each listing across available months
price_consistency <- data_1 %>%
  filter(!is.na(price)) %>% # throw away rows where price is missing
  group_by(id) %>% # group all rows of individual listings based on id
  summarise( # each listing id becomes a single row
    unique_prices = n_distinct(price), # Counts how many different prices a listing had
    consistent_price = first(price), # Grabs the first price it sees for that listing
    months_observed = n() # count how many total valid monthly data points exist for that specific list
  )

# Impute prices ONLY for listings that have a proven stable price history
# See how many listings have stable (unchanging) prices vs changing prices
table(price_consistency$unique_prices)
```

```
##
## 1 2 3 4 5 6 7 8
## 446 498 523 536 572 583 535 411
```

```
# Create a lookup table of ONLY the strictly stable listings
stable_price_lookup <- price_consistency %>%
  filter(unique_prices == 1) %>%
  select(id, consistent_price)

fully_missing_months <- missing_by_month %>%
  filter(pct_missing == 100) %>%
  pull(month_year)

# Impute prices safely using ifelse() to prevent type mismatch crashes
data_2 <- data_1 %>%
  left_join(stable_price_lookup, by = "id") %>%
  mutate(
    # If price is missing AND it matches a fully-missing-price month, pull from consistent_price
    price = ifelse(
      is.na(price) & month_year %in% fully_missing_months,
      consistent_price,
      price
    )
  ) %>%
  select(-consistent_price) # clean up temporary lookup column

# Check how many missing prices remain
print(sum(is.na(data_2$price)))
```

```
## [1] 10905
```

So 509 prices were filled as they were the same across every single other months they appeared in (id)

Still has 10158 missing prices

Manage price outliers/extreme values Inspect the extreme rows

```
# Check the top 20 most expensive listings using summary-file columns
top_expensive <- data_2 %>%
  filter(!is.na(price)) %>%
  arrange(desc(price)) %>%
  select(id, name, price, room_type, minimum_nights) %>%
  head(20)

print(top_expensive)
```

```
## # A tibble: 20 x 5
##       id name                                price room_type minimum_nights
##   <dbl> <chr>                                <dbl> <chr>          <dbl>
## 1 9.15e17 La Maison de Manu Waiata - Akaroa Hol~ 43654 Entire h~         1
## 2 5.94e17 Seaview Outlook - Wainui Holiday Home 43654 Entire h~         2
## 3 1.00e18 Pacific Panorama - Christchurch Holid~ 43654 Entire h~         2
## 4 1.41e18 Central Cutie - Christchurch City Cot~ 43654 Entire h~         2
## 5 5.94e17 Seaview Outlook - Wainui Holiday Home 43188 Entire h~         2
## 6 9.15e17 La Maison de Manu Waiata - Akaroa Hol~ 43188 Entire h~         1
## 7 1.00e18 Pacific Panorama - Christchurch Holid~ 43188 Entire h~         2
## 8 1.41e18 Central Cutie - Christchurch City Cot~ 43178 Entire h~         2
## 9 1.29e18 Groups wanted CHCH CBD 3bed 3 bath wi~ 22971 Entire h~         1
## 10 2.99e 7 Central Luxurious House             Fre~ 11865 Entire h~         1
## 11 1.85e 7 The East Penthouse                  ~ 7102 Private ~         1
## 12 1.85e 7 The East Penthouse                  ~ 6370 Private ~         1
## 13 1.85e 7 The East Penthouse                  ~ 6277 Private ~         1
## 14 1.85e 7 The East Penthouse                  ~ 6277 Private ~         1
## 15 1.85e 7 The East Penthouse                  ~ 6277 Private ~         1
## 16 1.85e 7 The East Penthouse                  ~ 6277 Private ~         1
## 17 1.85e 7 The East Penthouse                  ~ 5000 Private ~         1
## 18 1.85e 7 The East Penthouse                  ~ 5000 Private ~         1
## 19 1.50e18 Elegant Mansion for Weddings, Busines~ 3029 Entire h~         2
## 20 1.50e18 Elegant Mansion for Weddings, Busines~ 3029 Entire h~         2
```

Rows 1-8 -> likely error (checking latitude/altitude), doesn't make sense to have that much a night Rows 10-15 -> also likely error. priced >5000 per night in a private room doesn't make sense Rows 16-20.. maybe? entire home

Keep rows 1-15 will cause statistical bias into analysis. It distorts the mean and standard deviation DECISION: Filter these 15 rows (note: filter() keeps every row where the condition evaluates to TRUE, and throws away anything that evaluates to FALSE)

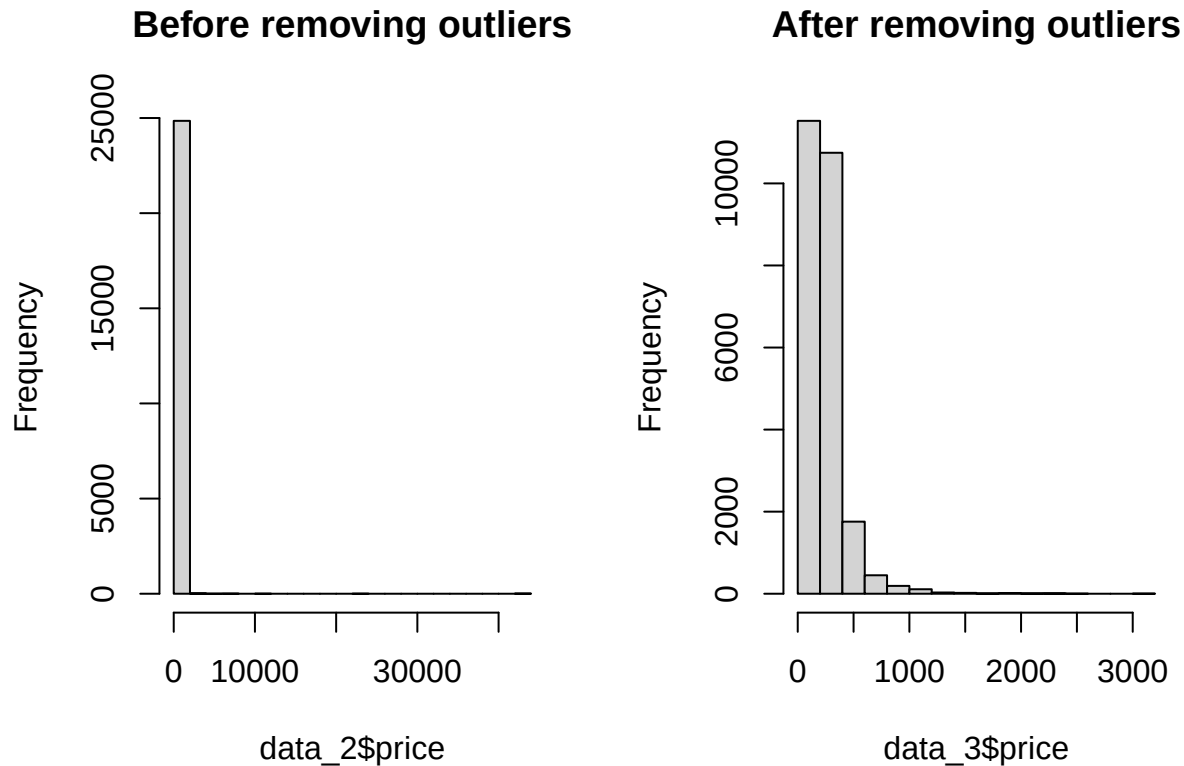
```
# Clean the dataset by removing anything $5,000 or higher (while keeping NAs. Otherwise R will remove NAs)
data_3 <- data_2 %>%
  filter(price < 5000 | is.na(price))

# Check how many rows were dropped
nrow(data_2) - nrow(data_3)
```

```
## [1] 18
```

Sanity check - compare histogram “before” and “after” remove outliers, to confirm the \$5000 cutoff actually removed tail of extreme values

```
par(mfrow = c(1, 2))
hist(data_2$price, main = "Before removing outliers")
hist(data_3$price, main = "After removing outliers")
```



```
# 1. Check the new maximum price to ensure the errors are truly gone
max(data_3$price, na.rm = TRUE)
```

```
## [1] 3029
```

```
# 2. Calculate clean summary statistics for prices
price_summary <- data_3 %>%
  filter(!is.na(price)) %>%
  summarise(
    mean_price = mean(price),
    median_price = median(price),
    min_price = min(price),
    max_price = max(price)
  )

print(price_summary)
```

```
## # A tibble: 1 x 4
```

```
##   mean_price median_price min_price max_price
##   <dbl>         <dbl>         <dbl>         <dbl>
## 1     243.           210           27           3029
```

What to do with remaining PRICE NAs? - leave them as NAs for now as not sure what is happening downward

License 100% NAs - no universal registration rules so hosts didn't have to register for official registration code to list their property -> DECISION DROP license column

```
# Drop the useless license column
df_cleaned <- data_3 %>%
  select(-any_of("license"))

# check columns
colnames(df_cleaned)
```

```
## [1] "id" "name"
## [3] "host_id" "host_name"
## [5] "neighbourhood_group" "neighbourhood"
## [7] "latitude" "longitude"
## [9] "room_type" "price"
## [11] "minimum_nights" "number_of_reviews"
## [13] "last_review" "reviews_per_month"
## [15] "calculated_host_listings_count" "availability_365"
## [17] "number_of_reviews_ltm" "month_year"
## [19] "scrape_date" "days_since_review"
```

check if reviews_per_month and last_review are related

```
# Check if the exact same rows are missing values for both columns
identical(is.na(df_cleaned$reviews_per_month), is.na(df_cleaned$last_review))
```

```
## [1] TRUE
```

if property has 0 reviews, both reviews_per_month, and last_review are missing.

```
# 1. Check if there are any literal 0 values already in reviews_per_month
sum(df_cleaned$reviews_per_month == 0, na.rm = TRUE)
```

```
## [1] 0
```

```
# 2. Check if there are any non-NA values in last_review when reviews_per_month is NA (or vice versa)
df_cleaned %>%
  filter(is.na(reviews_per_month) != is.na(last_review)) %>%
  select(id, name, reviews_per_month, last_review)
```

```
## # A tibble: 0 x 4
## # i 4 variables: id <dbl>, name <chr>, reviews_per_month <dbl>,
## #   last_review <date>
```

There are zero listings.. every single listing either has a positive review or is missing (NA). So reviews_per_month and last_review are 100% structurally locked

DECISION: takes every missing value (NA) in the reviews_per_month column and turns it into a literal 0

```
# Finalize the review columns treatment based on structural verification
df_cleaned <- df_cleaned %>%
  mutate(
    # Recode NA to 0 for numerical analysis (since we proved 0 doesn't otherwise exist)
    reviews_per_month = replace_na(reviews_per_month, 0)
    # last_review remains NA, perfectly justified by the zero-review status
  )

# 1. Check how many NA values remain in reviews_per_month (should be 0)
sum(is.na(df_cleaned$reviews_per_month))
```

```
## [1] 0
```

```
# 2. Check how many 0s are now in reviews_per_month (should match your previous missing count, 2627)
sum(df_cleaned$reviews_per_month == 0)
```

```
## [1] 3224
```

```
# check columns
colSums(is.na(df_cleaned))
```

```
##          id          name
##          0          0
##      host_id      host_name
##          0          1
## neighbourhood_group neighbourhood
##          0          0
##      latitude      longitude
##          0          0
##      room_type      price
##          0      10905
##      minimum_nights  number_of_reviews
##          37          0
##      last_review      reviews_per_month
##      3224          0
## calculated_host_listings_count  availability_365
##          0          0
##      number_of_reviews_ltm      month_year
##          0          0
##      scrape_date      days_since_review
##          0      3224
```

The Review-Related Columns (2,619 NAs) last_review: 2,619 last_review_date: 2,619 days_since_review: 2,619 These three columns are locked together Decision to leave them as NA since can't calculate "days since a review" if there has never been a review.

Price column: 10158 NAs

minimum_nights: 37 NAs

TO DECIDE: MINIMUM_NIGHTS NAs? impute with median : min-1, max-365, median-1, mean - 1.788

COLUMNS to drop: neighbourhood_group only has 1 unique value so can't help compare anything

```
# Check is the right column
head(df_cleaned$neighbourhood_group)
```

```
## [1] "Christchurch City" "Christchurch City" "Christchurch City"
## [4] "Christchurch City" "Christchurch City" "Christchurch City"
```

DECISION: DROP neighbourhood_group column

```
# Drop the zero-variance column (replace with the exact column name, e.g., neighbourhood_group)
df_cleaned <- df_cleaned %>%
  select(-neighbourhood_group)
colnames(df_cleaned)
```

```
## [1] "id" "name"
## [3] "host_id" "host_name"
## [5] "neighbourhood" "latitude"
## [7] "longitude" "room_type"
## [9] "price" "minimum_nights"
## [11] "number_of_reviews" "last_review"
## [13] "reviews_per_month" "calculated_host_listings_count"
## [15] "availability_365" "number_of_reviews_ltm"
## [17] "month_year" "scrape_date"
## [19] "days_since_review"
```

```
#check to see if last_review and last_review_date contain the exact same data
# Check if both columns are identical element-by-element
identical(df_cleaned$last_review, df_cleaned$last_review_date)
```

```
## Warning: Unknown or uninitialised column: 'last_review_date'.
```

```
## [1] FALSE
```

DECISIONS: Drop: host_name : irrelevant to analysis, and is personal identifiable information

```
df_cleaned <- df_cleaned %>%
  select(-c(host_name))
colnames(df_cleaned)
```

```
## [1] "id" "name"
## [3] "host_id" "neighbourhood"
## [5] "latitude" "longitude"
## [7] "room_type" "price"
## [9] "minimum_nights" "number_of_reviews"
## [11] "last_review" "reviews_per_month"
## [13] "calculated_host_listings_count" "availability_365"
## [15] "number_of_reviews_ltm" "month_year"
## [17] "scrape_date" "days_since_review"
```

```
#Check if any listing has more LTM reviews than total reviews (should output 0 rows)
df_cleaned %>%
  filter(number_of_reviews_ltm > number_of_reviews)
```

```
## # A tibble: 0 x 18
## # i 18 variables: id <dbl>, name <chr>, host_id <dbl>, neighbourhood <chr>,
## #   latitude <dbl>, longitude <dbl>, room_type <chr>, price <dbl>,
## #   minimum_nights <dbl>, number_of_reviews <dbl>, last_review <date>,
## #   reviews_per_month <dbl>, calculated_host_listings_count <dbl>,
## #   availability_365 <dbl>, number_of_reviews_ltm <dbl>, month_year <chr>,
## #   scrape_date <date>, days_since_review <dbl>
```

Check duplicates

```
# Count how many fully duplicated rows exist
sum(duplicated(df_cleaned))
```

```
## [1] 0
```

```
# Count how many duplicate IDs exist
sum(duplicated(df_cleaned$id))
```

```
## [1] 31518
```

```
#See which specific IDs are duplicated and how many times
df_cleaned %>%
  count(id) %>%
  filter(n > 1)
```

```
## # A tibble: 4,052 x 2
##       id       n
##   <dbl> <int>
## 1  204050     2
## 2  319770    11
## 3  737452    11
## 4 1004252    11
## 5 1007247    11
## 6 1097487    11
## 7 1429281    11
## 8 1461771    11
## 9 1798383    11
## 10 1837611    11
## # i 4,042 more rows
```

```
#check if these duplicate IDs are just the same property showing up in different months
# Pick one of the duplicated IDs from your output and see its monthly breakdown
df_cleaned %>%
  filter(id == 204050) %>% # replace 204050 with one of your actual duplicated IDs
  select(id, month_year, price, availability_365)
```

```
## # A tibble: 2 x 4
##       id month_year price availability_365
##   <dbl> <chr>      <dbl>          <dbl>
## 1 204050 2025-11      98            364
## 2 204050 2025-10      95            365
```

The same listing IDs appear multiple times because Inside Airbnb scraped them across different months, capturing how their prices and availability dynamically changed over time.

This means they are not accidental duplicate errors. They are time-series records.

```
# 1. Quick check of column types (ensure prices/lat/long are numeric, dates are proper, etc.)
glimpse(df_cleaned)
```

```
## Rows: 35,778
## Columns: 18
## $ id                <dbl> 319770, 737452, 1004252, 1007247, 10974~
## $ name              <chr> "Gateway to the South Island", "Mumford~
## $ host_id           <dbl> 1639584, 3851282, 5523780, 4765250, 499~
## $ neighbourhood     <chr> "Innes Ward", "Banks Peninsula Ward", "~
## $ latitude          <dbl> -43.50961, -43.82343, -43.61889, -43.53~
## $ longitude         <dbl> 172.6555, 172.9464, 172.6531, 172.6070,~
## $ room_type         <chr> "Private room", "Entire home/apt", "Ent~
## $ price             <dbl> 196, 394, 183, 305, 189, 69, 337, 166, ~
## $ minimum_nights    <dbl> 1, 2, 2, 2, 1, 5, 3, 2, 2, 2, 1, 2, 2, ~
## $ number_of_reviews <dbl> 3, 102, 227, 119, 230, 205, 16, 181, 98~
## $ last_review       <date> 2015-01-09, 2026-04-11, 2026-04-06, 20~
## $ reviews_per_month <dbl> 0.02, 0.68, 1.44, 0.79, 2.05, 1.33, 0.1~
## $ calculated_host_listings_count <dbl> 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 3, 1, 1, ~
## $ availability_365   <dbl> 364, 194, 173, 304, 295, 309, 45, 344, ~
## $ number_of_reviews_ltm <dbl> 0, 3, 10, 5, 0, 23, 0, 11, 15, 6, 54, 7~
## $ month_year        <chr> "2026-04", "2026-04", "2026-04", "2026--
## $ scrape_date       <date> 2026-04-30, 2026-04-30, 2026-04-30, 20~
## $ days_since_review <dbl> 4129, 19, 24, 68, 423, 11, 696, 68, 72,~
```

```
# 2. Confirm total rows remaining
nrow(df_cleaned)
```

```
## [1] 35778
```

Summary:

1. price column Initially had 10,667 missing values, with 100% missing values for the 3 months: Dec 2025–Feb 2026. Imputed 509 prices where the listing had the same price across all other months. Removed 15 extreme outliers with prices > \$5,000, which were likely errors. Before: 28,795 rows After: 28,780 rows Question: What should we do with the remaining 10,158 missing values? The price column is still right-skewed.
2. days_since_review column. Added days_since_review as instructed in Week 5. Initially, scrape_date was set to the first day of each month (YYYY-MM-01), resulting in 9,418 negative values. Updated scrape_date to use the actual day the listing was scraped and recalculated days_since_review. There are now 1,745 negative values, likely because some reviews were posted after the scrape date. Question: Do we need this column? If so, how should we handle the negative values?

3. Removed columns: license – removed because it was 100% missing. neighbourhood_group – removed because all values were Christchurch_city, so it provides no useful comparison. host_name – removed because it contains personally identifiable information and is not relevant to the analysis.
4. reviews_per_month and last_review: Converted 2,619 missing values in reviews_per_month to 0, since listings with no reviews have both reviews_per_month and last_review recorded as NA. Left last_review NAs unchanged for now, since replacing them with 0 would not make sense for a date variable.
5. minimum_nights: Has 37 missing values. Current statistics: min = 1, median = 1, mean = 1.788, max = 365. Have left as it is. Depend on analysis later - impute missing values using median (1) or leave as it is?
6. Other findings: There were no exact duplicates. The same listing IDs appear multiple times because Inside Airbnb scraped them across different months, capturing how their prices and availability changed over time. So they are time-series records.

Before cleaning- number of rows: 28795, columns: 19 After cleaning: 28780, columns: 18

(Note: Cleaned dataset: df_cleaned)

```
write.csv(df_cleaned, file = "../out/airbnb_cleaned_week_8.csv", row.names = FALSE)
```